GLRMask: Constrained Decoding with Boolean Array Weighted Automata

Isaac Breen

October 14, 2026

Abstract

Grammar-constrained decoding (GCD) forces LLM outputs to conform to a specified grammar. During sampling, GCD applies a mask to the model's next-token distribution, ruling out tokens that would leave no grammar-valid continuation. For context-free grammars, token admissibility under this mask depends on the parser state. Existing CFG-capable systems commonly resolve this dependence by exploring parser behavior online during mask generation. For delimiter-rich JSON-schema workloads this online work is often small enough that optimized systems are already practical; it becomes a sharper issue for tail predictability and for grammars whose lexer/parser interactions are less JSON-like. This paper presents GLRMask, a GCD system that compiles the parser-state-dependent part of token admissibility ahead of time. Given a grammar and a vocabulary, GLRMask constructs a deterministic weighted automaton, the *Parser DWA*, whose input symbols are parser-stack symbols and whose weights are Boolean masks over lexer-state/token pairs. At runtime, mask generation traverses the graph-structured parser stack through this automaton one symbol at a time, with each transition's Boolean weight narrowing a running set of allowed tokens as it is taken. The traversal treats the parser stack as read-only. State-mutating parser actions are confined to *commit*, which advances the parser with the sampled token. On 1,734,930 positions in 11,277 complete valid JSONSchemaBench trajectories accepted by all four compared engines, Static GLRMask has median Time Between Masks of 2.61 μs, p99 of 12.15 μs and maximum of 283.07 μs, compared with 10.63 μs, 289.92 μs and 6.93 ms for llguidance. These observations are from one unpinned native wall-clock AWS run with overlapping checkpoint work; retained telemetry does not certify latency isolation.

1 Introduction

Large language models are now routinely asked to produce output that must satisfy hard structural constraints: JSON objects matching a schema, tool calls with typed arguments, SQL or configuration fragments, programming-language expressions, and domain-specific command languages. Grammar-constrained decoding (GCD) enforces such constraints during generation, guaranteeing that the output conforms to a target grammar. Rather than sampling from the model's distribution directly, the decoder first masks out every next token that would lead to a grammatical dead end, then samples from what survives, so generation never enters a prefix from which no valid completion exists.

For a target byte language $L \subseteq \mathcal{B}^*$, a byte prefix u, and a tokenizer vocabulary $\mathcal{V}$, the mask contains exactly those tokens whose bytes leave at least one valid completion:

$$Mask_L(u) = \{v \in \mathcal{V} : \exists w \in \mathcal{B}^*.\ u\beta(v)w \in L\},$$

where $\beta(v)$ is the byte string of token v. At each step the decoder computes $Mask_L(u)$, samples from the model distribution restricted to that set, and commits the chosen token.

For regular constraints, this is simple. Before decoding, build a mask for each automaton state. Run each vocabulary token from that state; if the run avoids the dead state, include the token in

that state's mask. A vocabulary trie keeps this pass efficient by avoiding repeated work on shared prefixes. At runtime, the decoder just uses the mask for its current state.

For context-free grammars, a lexer is not enough. It can recognize terminal structure, but it cannot represent the grammar or summarize parser context. A token may admit no completed terminal, one completed terminal, or several completed terminals from the current lexer state; any completed terminals must also be compatible with the current parser stack. Under lexical or grammatical ambiguity, the same token can induce many lexer paths, terminal sequences, and parser continuations.

Existing CFG-capable decoders commonly answer this query by online search. During mask generation, they run candidate tokens through the lexer and test the parser actions or continuations those tokens may induce. This puts hypothetical parser execution in the token-generation loop. On delimiter-rich JSON workloads, optimized online systems can make this cost small enough for ordinary serving. On less regular grammars, and in the tail even on JSON-like workloads, the cost still depends on lexer ambiguity and parser ambiguity.

This paper presents GLRMask, a grammar-constrained decoding system that compiles the parser-dependent part of this search ahead of time.[1] Given a grammar and tokenizer vocabulary, GLRMask constructs a deterministic weighted automaton over LR stack symbols whose weights are token masks. At runtime, mask generation runs this automaton on the current parser stack, one symbol at a time. The returned mask is the intersection of the weights encountered along this run. Parser actions are not explored during this traversal. They are confined to commit, after a token has actually been sampled.

The result is an exact CFG-constrained decoder with predictable per-token latency. It handles vocabulary tokens that cross grammar-terminal boundaries while preserving the standard mask/sample/commit interface. The parser still determines which tokens are admissible, but the parser-dependent search for hypothetical tokens is compiled ahead of time. This is most valuable when the grammar is reused, when worst-case mask latency matters, or when the grammar is not one of the delimiter-separated JSON schemas for which online systems are already highly optimized.

This paper makes the following contributions:

1. **A stack-read characterization of exact CFG token admissibility**, separating the lexer/token relation from the parser-stack relation and showing that mask queries require only the regular read projections of composed terminal stack effects.

2. **The Parser DWA**, a compact compile-time construction that shares the read-prefix languages of all lexer-state/token pairs in a single deterministic weighted automaton over stack symbols, built by effect substitution, write/read cancellation, read projection, and determinization.

3. **A replay-free exact runtime** that evaluates the compiled automaton over graph-structured parser stacks, combines masks across branches and merges, handles longest-match lexing and end-of-sequence exactly, and applies parser transitions only at commit.

4. **An empirical evaluation** demonstrating microsecond-scale, tightly bounded mask latency on realistic structured-output workloads.

[1] PSC [14] also compiles stack-dependent mask computation ahead of time, targeting deterministic context-free grammars. Section 11 discusses the relationship between the approaches.

2 Mask Semantics

This section makes the mask semantics explicit. It defines the language-level mask, the runtime configuration set, and the MASK and COMMIT operations.

2.1 Language-Level Mask

Let L be the language recognised by the grammar, and let $\beta(v)$ be the byte string for token v. After prefix u, token v is valid exactly when $u\beta(v)$ can still be completed to a string in L:

$$\mathrm{Mask}_L(u) = \{\, v \in \mathcal{V} : \exists w,\ u\,\beta(v)\,w \in L \,\}. \tag{1}$$

The following is an efficient, exact implementation of this one set.

2.2 Runtime State

The runtime never stores the consumed prefix u; it stores a finite set of possible configurations instead. Let $\mathcal{T}$ be the grammar terminals, Q_{lex} the lexer states, Q_{LR} the parser stack symbols, and $\mathcal{E}$ the set of longest-match exclusion maps. A *configuration set* is a finite set of runtime configurations,

$$C \subseteq Q_{\mathrm{lex}} \times Q_{\mathrm{LR}}^{*} \times \mathcal{E},$$

where each configuration is a triple (q, σ, e):

- $q \in Q_{\mathrm{lex}}$ is the current lexer state;
- $\sigma = p_0 p_1 \cdots p_m \in Q_{\mathrm{LR}}^{*}$ is the current parser stack, written top-first, so p_0 is the top-of-stack state;
- $e : Q_{\mathrm{lex}} \rightharpoonup 2^{\mathcal{T}}$ is a longest-match exclusion map in $\mathcal{E}$. An entry $t \in e(q')$ declares that this configuration must be discarded if terminal t matches while scanning the next fragment from lexer state q'.

2.3 Runtime Mask

Token validity over a configuration factors into a lexer side and a parser side. For a single configuration, token v is admissible exactly when both conditions hold:

1. v passes the longest-match exclusion check, $\mathrm{ExclOK}(e, \beta(v))$.
2. Some terminal sequence r induced by scanning $\beta(v)$ from lexer state q can be parsed from stack σ.

Equivalently,

$$\begin{aligned} \mathrm{Adm}(q, \sigma, e; v) \quad \Longleftrightarrow \quad & \mathrm{ExclOK}(e, \beta(v)) \\ & \wedge\, \exists r \in \mathrm{Lex}(q, \beta(v)) : \mathrm{Parseable}(r, \sigma). \end{aligned} \tag{2}$$

The three relations here are defined and compiled in §5:

- $\mathrm{Lex}(q, b) \subseteq \mathcal{T}^{*}$: the complete terminal sequences obtained by scanning fragment b from lexer state q (§3);
- $\mathrm{ExclOK}(e, b)$: whether fragment b respects the longest-match exclusions in e (§3.4);
- $\mathrm{Parseable}(r, \sigma)$: whether terminal sequence r can be parsed from stack σ (§4.6).

For a configuration set C reached after consuming prefix u, the runtime mask collects the tokens admitted by at least one configuration,

$$\mathrm{Mask}_{\mathrm{rt}}(C) = \{\, v \in \mathcal{V} : \exists (q, \sigma, e) \in C,\ \mathrm{Adm}(q, \sigma, e; v) \,\},$$

The rest of the paper maintains the invariant that this runtime mask agrees with the specification:

$$\mathrm{Mask}_{\mathrm{rt}}(C) = \mathrm{Mask}_L(u). \tag{3}$$

§5 builds the automaton that makes $\mathrm{Mask}_{\mathrm{rt}}$ a walk; §6 gives the walk and shows that COMMIT preserves (3) (§7).

2.4 Mask and Commit

With this invariant fixed, each decoding step has two operations.

1. MASK evaluates the Parser DWA on the parser stacks and combines the weights encountered along the walk into a vocabulary mask.
2. COMMIT, after the sampler chooses a token, scans the token bytes, emits completed terminals, applies the compiled stack effect for each emitted terminal, and updates longest-match exclusions. The same operation can advance by any byte fragment, which is what prefill needs.

The Parser DWA, its input labels (parser stack symbols), and its weights (masks over lexer-state/token pairs) are built once by the compiler of §5. In the evaluated implementation the stack symbols are LR parser states, but MASK only reads stack symbols and combines weights; it never replays parser transitions for candidate tokens.

Example 2.1 (Running example)**.** We thread one small grammar through the method. It accepts balanced brackets nested around a single `x`, so `[x]`, `[[x]]`, and `[[[x]]]` are in the language:

```
start ::= "[" inner "]",    inner ::= "x" | "[" inner "]".
```

The terminals are `[`, `]`, and `x`. The vocabulary contains the single-terminal tokens `[`, `]`, `x` and the two-terminal tokens `[[`, `]]`, `x]`, `[x`; the two-terminal tokens are what make within-token parser effects visible. The masks below are produced by the implementation:

after prefix	admissible tokens
ϵ (start)	`[`, `[[`, `[x`
`[`	`[`, `x`, `[[`, `x]`, `[x`
`[[`	`[`, `x`, `[[`, `x]`, `[x`
`[x`	`]`

Every mask depends on the parser stack, not just the lexer: `]` is rejected at the start and after `[` (an `inner` is still required) but is the only admissible token after `[x`, and `]]` is always rejected here because no prefix reaches bracket depth two with its inner already closed. §5 and §6 revisit this example to show how the compiled automaton produces these masks.

3 Lexer

The lexer side records how token bytes scan into grammar-terminal sequences. It first fixes a deterministic lexer model, then defines execution on a byte fragment, then accounts for partial terminal matches and longest-match exclusions.

3.1 Model

Let $\mathcal{B}$ be the byte alphabet. We model the lexer as a total DFA

$$\mathcal{A}_{\text{lex}} = (Q_{\text{lex}}, \mathcal{B}, \delta, q_0, \bot, A),$$

where $q_0 \in Q_{\text{lex}}$ is the initial state, $\bot \in Q_{\text{lex}}$ is the rejecting state, $\delta : Q_{\text{lex}} \times \mathcal{B} \to Q_{\text{lex}}$ is total, $\delta(\bot, b) = \bot$, and $A : Q_{\text{lex}} \to 2^{\mathcal{T}}$ labels each lexer state with the terminals accepted there, with $A(\bot) = \emptyset$. For a byte fragment $b = b_1 \cdots b_n$, the run from state q is $q^{(0)} = q,\ q^{(i)} = \delta(q^{(i-1)}, b_i)$, and we write $\delta^*(q, b) = q^{(n)}$.

We adopt a clean-start convention for q_0, under which no ordinary lexer transition enters q_0: it is reached only by lexer initialization or by explicit reinitialization immediately after a terminal match. Thus q_0 marks terminal boundaries. This convention can always be enforced by splitting the raw DFA start state.

3.2 Execution

The match operation records the terminals accepted by prefixes of this one run, together with the final lexer state. Define

$$\text{Match}(q, b) = (\mu, q^{(n)}), \qquad \mu(t) = \max\{\, i : 1 \le i \le n,\ t \in A(q^{(i)}) \,\},$$

where $\mu : \mathcal{T} \rightharpoonup \{1, \ldots, n\}$ is partial, defined when the set is nonempty. Thus $\mu(t)$ is the longest prefix of b that matches terminal t from state q. If the run reaches $\bot$, no later state accepts a terminal, since $\bot$ is absorbing and $A(\bot) = \emptyset$. The continuation relation is defined by reachability:

$$\text{CanMatch}(q) = \{\, t \in \mathcal{T} : \exists b \in \mathcal{B}^*,\ t \in A(\delta^*(q, b)) \,\},$$

the terminals that can be completed from lexer state q, allowing the empty continuation.

A byte fragment can contain several completed terminal boundaries, or end in the middle of a terminal match. We record both possibilities with a branching scan relation $\text{Scan}(q, b) \subseteq \mathcal{T}^* \times Q_{\text{lex}}$. It is the least relation satisfying two clauses. First, if scanning all of b from q does not reject, the fragment may be left unemitted:

$$(\epsilon, \delta^*(q, b)) \in \text{Scan}(q, b) \qquad \text{if } \delta^*(q, b) \neq \bot.$$

Second, let $\text{Match}(q, b) = (\mu, -)$. If $t \in \text{dom}(\mu)$, $\mu(t) = i$, and $b = cd$ with $|c| = i$, then for every $(r, q') \in \text{Scan}(q_0, d)$,

$$(tr, q') \in \text{Scan}(q, b).$$

This clause emits terminal t, resets the lexer to q_0, and continues scanning the remaining suffix. The recursion is finite because every emitted terminal consumes a nonempty prefix of b.

3.3 Partial Matches

$\text{Scan}(q, b)$ records outcomes, each a sequence r of terminals completed inside the fragment together with the lexer state q' left at the fragment boundary. Consider a single match $(r, q') \in \text{Scan}(q, b)$. If scanning ends at $q' = q_0$, the scanned terminal sequence r is already complete; if the parser accepts r, the bytes b are valid next. If scanning ends in a non-boundary state $q' \neq q_0$, it has stopped in the middle of a partial terminal match that straddles the fragment boundary. In this case it is not

enough for the parser to accept r: it must also accept rt for some $t \in \mathrm{CanMatch}(q')$ that could complete the partial match. The *completed* lexer relation used in Equation (2) is therefore

$$\mathrm{Lex}(q, b) = \bigcup_{(r,q') \in \mathrm{Scan}(q,b)} \begin{cases} \{r\}, & q' = q_0, \\ \{rt : t \in \mathrm{CanMatch}(q')\}, & q' \neq q_0. \end{cases}$$

Each $r \in \mathrm{Lex}(q, b)$ is a complete terminal sequence that scanning b from q may yield, possibly including a terminal that completes a partial match when b ends mid-terminal, so it is ready for parser-stack checking.

In Example 2.1 the terminals are single characters, so scanning is immediate: the token `[[` scans from q_0 into the terminal sequence `[`[, and `x]` into `x]`. A two-terminal token thus presents two grammar terminals to the parser side inside one decoding step.

3.4 Longest-Match Exclusions

We adopt a longest-match convention for the lexer: a terminal is emitted only when the same match can no longer be extended. For example, an identifier-like match `ab` remains tentative while a longer match such as `abc` is still possible.

GLRMask handles tentative terminal boundaries by splitting the runtime configuration. At a tentative boundary where terminal t has matched but the same match can continue from lexer state q', the branch that emits t records $t \in e(q')$. This declares the branch invalid if a later fragment scanned from q' encounters t, which is exactly the case in which the earlier emission would violate longest-match lexing. For a candidate fragment b, the longest-match exclusion check is

$$\mathrm{ExclOK}(e, b) \quad \Longleftrightarrow \quad \forall (q', X) \in e : \ \mathrm{dom}(\mu_{q',b}) \cap X = \emptyset,$$

where $\mathrm{Match}(q', b) = (\mu_{q',b}, -)$. A branch passes the check exactly when no excluded terminal is matched by the fragment from the lexer state at which the exclusion was recorded. How e is updated is described in §6.3.

4 Parser Stack Effects

The parser side records, for each terminal, how consuming that terminal rewrites the parser stack. This section fixes the stack-effect interface, shows how the implementation derives effects from GLR tables, describes the corresponding real-time GPDA interface, and then composes and projects effects into the read-prefix languages used by Mask.

4.1 Stack Effects

For each terminal t, the parser side records the stack-prefix rewrites by which t can be consumed. These rewrites form a stack-effect language E_t.

Let Γ be the parser stack alphabet, write stacks top-first, and define

$$\Gamma^{\pm} := \{\, p^-, p^+ : p \in \Gamma \,\}.$$

A word

$$u^- v^+ \in E_t$$

records a rewrite with read prefix u and replacement prefix v. The read prefix is marked negatively,

$$u^- := p_1^- \cdots p_m^-,$$

and the replacement prefix is marked positively, in push order,

$$v^+ := q_n^+ \cdots q_1^+.$$

The positive block is reversed because stacks are written top-first: lower symbols of v are pushed before its top symbols. Thus $u^- v^+$ denotes

$$u\alpha \mapsto v\alpha \qquad (\alpha \in \Gamma^*),$$

where the top prefix u is replaced by v and the remaining stack is unchanged.

4.2 Stack Effects from GLR Tables

In the implementation, the source parser is GLR, with stack symbols Q_{LR} given by an IELR(1) table. A shift on terminal t from state q to state q' contributes the signed word

$$q^- q^+ q'^+ \in E_t,$$

which represents the stack rewrite

$$q\alpha \mapsto q' q\alpha.$$

A reduction by $A \to \beta$, with $|\beta| = k$, does not itself consume t; it is selected while t is the lookahead terminal. If the stack begins with k states above a predecessor state p, the reduction pops those k states, exposes p, follows the goto entry $g = \mathrm{goto}(p, A)$, and pushes g. Thus, when $\sigma = q_k \cdots q_1 p\alpha$,

$$q_k \cdots q_1 p\alpha \mapsto gp\alpha.$$

The predecessor p is read because it determines the goto state, but it is not removed from the final stack. For a terminal t, E_t contains the signed words obtained by taking a finite sequence of reductions chosen with lookahead t, followed by a shift on t. It describes exactly which stack prefixes allow terminal t to be consumed, and what stack prefix replaces them afterward.

4.3 Stack Effects from ϵ-Free GPDAs

The GLR table is one source of terminal stack effects; an ϵ-free generalized pushdown automaton gives the same information directly. Here ϵ-free means that every transition consumes one grammar terminal. Let

$$M = (P, \mathcal{T}, \Gamma, \Delta, p_0, \gamma_0, F)$$

be such a GPDA, with finite transition relation

$$\Delta \subseteq P \times \mathcal{T} \times \Gamma^* \times P \times \Gamma^*.$$

A transition $(p, t, u, p', v) \in \Delta$ consumes terminal t, changes control from p to p', and replaces stack prefix u by v:

$$(p, u\alpha) \mapsto (p', v\alpha).$$

Equivalently, we can fold the control state into the stack alphabet and view the transition as a one-terminal stack effect. We require each reachable instance of the transition to preserve completability: whenever $(p, u\alpha)$ is reachable, the target $(p', v\alpha)$ must be able to reach acceptance.

4.4 Stack-Effect Automata

A stack-effect automaton for terminal t is a finite automaton over $\Gamma^{\pm}$ whose accepted language is E_t. The compiler represents each stack-effect language by such an automaton.

4.5 Composition

The language E_t describes the effect of consuming a single terminal. To reason about a sequence of terminals, we compose their effects.

Composition is concatenation followed by cancellation:

$$p^+p^- \to \epsilon, \qquad p^+q^- \to \bot \quad (p \neq q).$$

A matching write-read pair cancels; a mismatch fails.

For

$$x = u^-v^+, \qquad y = w^-z^+,$$

write $\preceq$ for prefix order. Composition compares the written prefix v with the read prefix w:

$$x \odot y := \begin{cases} u^-a^+z^+ & v = wa, \\ u^-a^-z^+ & w = va, \\ \bot & v \not\preceq w \wedge w \not\preceq v. \end{cases}$$

The residual a is positive when it was written by x, and negative when it must be read from the original stack.

For languages $E, F \subseteq (\Gamma^{\pm})^*$, define their composition by collecting all successful word compositions:

$$E \odot F := \{\, x \odot y : x \in E,\ y \in F \,\},$$

where failed compositions are omitted.

For a terminal sequence $r = t_1 \cdots t_k$,

$$E_r := E_{t_1} \odot \cdots \odot E_{t_k}.$$

For the empty sequence,

$$E_\epsilon := \{\epsilon\}.$$

Thus E_r is the language of net stack effects for consuming r.

4.6 Read Projection

While Commit needs to update the stack, Mask only needs to know whether a terminal sequence r can be parsed from the current stack. We therefore keep only the read prefix:

$$\widehat{E}_r := \{\, u \in \Gamma^* : \exists v \in \Gamma^*,\ u^-v^+ \in E_r \,\}.$$

Thus r is parseable from stack σ exactly when σ begins with some $u \in \widehat{E}_r$:

$$\text{Parseable}(r, \sigma) \quad \iff \quad \exists u \in \widehat{E}_r,\ \exists \alpha :\ \sigma = u\alpha.$$

Since a token can induce several complete terminal sequences from the same lexer state, any parseable sequence suffices. Thus

$$\widehat{E}_{q,v} := \bigcup_{r \in \text{Lex}(q, \beta(v))} \widehat{E}_r.$$

This is the stack-prefix language that the Parser DWA must recognize for the pair (q, v).

The token `x]` in the running example (§2.1) shows why composition must happen before projection. It scans to the two-terminal sequence `x ]`, whose terminal effects contain internal cancellation that a mask query needs to ignore. Write b for the stack symbol witnessing the unmatched opening bracket already present when the token begins, p for the intermediate symbol obtained after parsing `x` as an `inner`, and c for the symbol left after the closing bracket has also been parsed. At this level of abstraction, the relevant terminal effects are

$$\begin{aligned} \texttt{x}: &\quad b^- b^+ p^+ \in E_{\texttt{x}}, \\ \texttt{]}: &\quad p^- b^- c^+ \in E_{\texttt{]}}. \end{aligned}$$

The names b, p, c are illustrative stack-effect operands, not concrete table state numbers. Composing the two terminal effects concatenates their signed words and cancels each pushed symbol when the next effect reads it:

$$\underbrace{b^- b^+ p^+}_{\texttt{x}}\ \underbrace{p^- b^- c^+}_{\texttt{]}} \xrightarrow{p^+ p^- \to \epsilon} b^- b^+ b^- c^+ \xrightarrow{b^+ b^- \to \epsilon} b^- c^+.$$

Thus $b^- c^+ \in E_{\texttt{x]}}$, and $\widehat{E}_{\texttt{x]}} = \{\, b \,\}$, closed under stack suffixes. The mask query does not need to remember the intermediate p, nor the copy of b that `x` read and restored. It only needs to test that b was already present. This is why the running example admits `x]` after prefix `[`: the token performs an internal parse-and-close sequence, but the surviving stack read is just the pre-existing open bracket.

5 Parser DWA Construction

§2 expressed admissibility using three ingredients: lexer scans, parser stack effects, and longest-match exclusions. The first two are compiled into the Parser DWA queried at runtime. The longest-match exclusion ingredient remains a runtime filter, handled by the lexer-match table in §6.2.2.

5.1 Boolean Weights

A weight is a Boolean mask

$$W \in K := \mathbb{B}^{Q_{\text{lex}} \times \mathcal{V}}.$$

Its coordinates are indexed by a lexer state q and a vocabulary token v:

$$W_{qv} \in \mathbb{B}, \qquad q \in Q_{\text{lex}},\ v \in \mathcal{V}.$$

The weight space K is an idempotent semiring: $\oplus$ is mask union and $\otimes$ is mask intersection. The operations are pointwise:

$$(W \oplus W')_{qv} = W_{qv} \vee W'_{qv}, \qquad (W \otimes W')_{qv} = W_{qv} \wedge W'_{qv}.$$

The zero element is the all-false mask, and the unit is the all-true mask.

5.2 Weighted Automata

Weighted automata multiply weights along a run. For the deterministic automata used here, the run is described by a partial transition function. Each transition on that path contributes a weight, and the final state contributes one more weight.

Formally, a DWA over alphabet Σ is a tuple

$$\mathcal{A} = (S, s_0, \tau, \varphi),$$

where S is a finite state set, $s_0 \in S$ is the start state, $\tau : S \times \Sigma \rightharpoonup S \times K$ is the partial weighted transition function, and $\varphi : S \to K$ assigns final weights. For a word $w = a_1 \cdots a_m$, write

$$s_0 \xrightarrow{a_1/W_1} s_1 \xrightarrow{a_2/W_2} \cdots \xrightarrow{a_m/W_m} s_m$$

when $\tau(s_{j-1}, a_j) = (s_j, W_j)$. Evaluation produces a weight $[\![\mathcal{A}]\!] : \Sigma^* \to K$,

$$[\![\mathcal{A}]\!](w) = W_1 \otimes \cdots \otimes W_m \otimes \varphi(s_m).$$

If the path is undefined the value is 0; for $w = \epsilon$ the empty product is the unit, so $[\![\mathcal{A}]\!](\epsilon) = \varphi(s_0)$. Since weights are masks over lexer-state/token pairs, a pair appears in $[\![\mathcal{A}]\!](w)$ exactly when it appears in every transition weight along the path and in the final weight.

5.3 Terminal DWA

The parser construction reasons about terminal sequences, but the model proposes vocabulary tokens. The Terminal DWA is the compressed weighted representation of this translation. Recall that, for lexer state q and token v,

$$\mathrm{Lex}(q, \beta(v)) \subseteq \mathcal{T}^*$$

is the set of complete terminal sequences obtained by scanning $\beta(v)$ from q. The automaton is defined by

$$[\![\mathcal{T}_{\mathrm{DWA}}]\!](r)_{qv} = 1 \quad \Longleftrightarrow \quad r \in \mathrm{Lex}(q, \beta(v)).$$

Thus, given a terminal sequence r, the Terminal DWA returns the mask of lexer-state/token pairs that can realize r exactly. It is compressed because the indexed family $\{\mathrm{Lex}(q, \beta(v))\}_{q,v}$ is represented by one weighted automaton.

5.4 Parser DWA

The Parser DWA is the runtime automaton walked over parser-stack prefixes. Its contract is

$$[\![\mathcal{P}_{\mathrm{DWA}}]\!](\rho)_{qv} = 1 \quad \Longleftrightarrow \quad \rho \in \widehat{E}_{q,v}.$$

Recall from §4.6 that the token-specific read-prefix language is

$$\widehat{E}_{q,v} := \bigcup_{r \in \mathrm{Lex}(q,\beta(v))} \widehat{E}_r.$$

Equivalently,

$$\begin{aligned} \rho \in \widehat{E}_{q,v} \quad &\Longleftrightarrow \quad \exists r : r \in \mathrm{Lex}(q, \beta(v)) \ \wedge \ \rho \in \widehat{E}_r \\ &\Longleftrightarrow \quad \exists r : [\![\mathcal{T}_{\mathrm{DWA}}]\!](r)_{qv} = 1 \ \wedge \ \rho \in \widehat{E}_r. \end{aligned}$$

The construction realizes this equivalence by substituting parser-side automata into Terminal-DWA paths. For each terminal t, let $\mathcal{A}_t$ be the stack-effect automaton for E_t. It accepts signed words $u^- v^+$, one for each stack rewrite represented in E_t. Substituting the family $\{\mathcal{A}_t\}_{t\in\mathcal{T}}$ into $\mathcal{T}_{\mathrm{DWA}}$ gives a weighted automaton over signed stack symbols. Each Terminal-DWA transition $x \xrightarrow{t/W} y$ is replaced by a fresh copy of $\mathcal{A}_t$: the copy's start and accepting states are identified with x and y, and every copied transition carries weight W. A path through the substituted automaton chooses a Terminal-DWA terminal sequence $r = t_1 \cdots t_k$, one accepting path in each $\mathcal{A}_{t_i}$, and carries the product of the Terminal-DWA weights on the original terminal path.

For example, suppose a Terminal-DWA path contains

$$x_0 \xrightarrow{t_1/W_1} x_1 \xrightarrow{t_2/W_2} x_2.$$

Let $\mathcal{A}_{t_1}$ and $\mathcal{A}_{t_2}$ be the stack-effect automata substituted for t_1 and t_2. Suppose $\mathcal{A}_{t_1}$ has an accepting path

$$s_{1,0} \xrightarrow{p^-} s_{1,1} \xrightarrow{q^+} s_{1,2},$$

and $\mathcal{A}_{t_2}$ has an accepting path

$$s_{2,0} \xrightarrow{q^-} s_{2,1} \xrightarrow{r^+} s_{2,2}.$$

Substitution identifies the start and accepting states of each copied automaton with the endpoints of the Terminal-DWA edge it replaces: $s_{1,0}$ with x_0, $s_{1,2}$ with x_1, $s_{2,0}$ with x_1, and $s_{2,2}$ with x_2. It also gives each copied edge the weight of the Terminal-DWA edge being replaced. Thus these accepting paths produce

$$x_0 \xrightarrow{p^-/W_1} s_{1,1} \xrightarrow{q^+/W_1} x_1 \xrightarrow{q^-/W_2} s_{2,1} \xrightarrow{r^+/W_2} x_2.$$

This path reads the signed word

$$p^- q^+ q^- r^+.$$

The adjacent q^+q^- pair is internal to the terminal sequence: q is pushed by the first terminal effect and immediately read by the second. Cancellation removes this pair:

$$q^+ q^- \to \epsilon.$$

Thus the whole signed word reduces to

$$p^- q^+ q^- r^+ \;\longrightarrow\; p^- r^+.$$

At the automaton level, the two cancelled edges are replaced by an ϵ-edge whose weight is the product of their weights:

$$x_0 \xrightarrow{p^-/W_1} s_{1,1} \xrightarrow{\epsilon/(W_1 \otimes W_2)} s_{2,1} \xrightarrow{r^+/W_2} x_2.$$

Since $\otimes$ is mask intersection, this ϵ-edge records exactly the lexer-state/token pairs that survive both sides of the cancelled stack action.

A mismatch is handled by the same local check, but discards the path. In the mirrored case, keep the first stack-effect path the same,

$$s_{1,0} \xrightarrow{p^-} s_{1,1} \xrightarrow{q^+} s_{1,2},$$

but suppose the second automaton instead reads a different symbol $h \neq q$:

$$s_{2,0} \xrightarrow{h^-} s_{2,1} \xrightarrow{r^+} s_{2,2}.$$

Substitution then produces

$$x_0 \xrightarrow{p^-/W_1} s_{1,1} \xrightarrow{q^+/W_1} x_1 \xrightarrow{h^-/W_2} s_{2,1} \xrightarrow{r^+/W_2} x_2,$$

which reads

$$p^- q^+ h^- r^+.$$

Now the boundary pair is q^+h^-. Since the pushed symbol and the read symbol differ, cancellation fails:

$$q^+h^- \to \bot \qquad (h \neq q),$$

so the whole substituted path is discarded.

After all cancellations, every surviving path has normal form

$$u^- v^+.$$

The read prefix u is the part of the parser stack that had to be present before r began; the write prefix v is the stack prefix left afterward. Mask only checks admissibility, so the construction projects each surviving path to its read side u, preserving the accumulated weight. Thus the substituted paths for r, after cancellation and projection, accept exactly the read-prefix language $\widehat{E}_r$.

If several substituted paths project to the same read-prefix path, their weights are unioned with $\oplus$. The result is a weighted automaton over parser-stack symbols whose paths read prefixes ρ, and whose weights record the (q, v) pairs for which some induced terminal sequence can be parsed from ρ. The Parser DWA is the deterministic runtime form of this weighted automaton; the boundedness assumptions that make it finite are stated in §8.

For any lexer-state/token pair (q, v), the read-prefix automaton for $\widehat{E}_{q,v}$ can be recovered from the Parser DWA by retaining exactly the transitions whose weights contain (q, v), and accepting exactly the states whose final weights contain (q, v). Thus a single Parser DWA stores the whole family of read-prefix automata: common states and transitions are represented once, while weights record which lexer-state/token pairs use each piece.

6 Runtime

The Parser DWA, the lexer-match table, and the stack-effect automata are all compiled once per grammar and vocabulary. The runtime stores compact parser configurations, answers Mask by walking graph-structured stacks through the Parser DWA, and advances state with Commit.

6.1 Runtime State

After each generated prefix, the runtime stores the configuration set C of §2.2 in a compact form: graph-structured parser stacks grouped by lexer state, with longest-match exclusions attached to stack branches. Formally it is a finite map

$$S : Q_{\text{lex}} \rightharpoonup \text{GSS}(Q_{\text{LR}}, \mathcal{E}),$$

from lexer states to graph-structured parser stacks. Each branch of a graph-structured stack represents one parser stack and carries the longest-match exclusion map $e \in \mathcal{E}$ for that branch. Enumerating all branches of all stacks in S, together with the lexer-state key of each map entry, gives back the configuration set

$$C \subseteq Q_{\text{lex}} \times Q_{\text{LR}}^* \times \mathcal{E}$$

of §2.2. Thus S compactly represents C. The graph structure shares common stack suffixes, but this sharing is purely representational: a mask query has the same semantics as checking each represented configuration separately.

6.2 Mask

A mask query computes $\mathrm{Mask}_{\mathrm{rt}}(C)$ from the compact runtime state S. For a single represented configuration (q, σ, e), Equation (2) says that token v is admissible when it passes the longest-match exclusion check and some terminal sequence induced by $\beta(v)$ is parseable from σ. By the definition of $\widehat{E}_{q,v}$, the parser-side condition is equivalently

$$\exists \rho, \alpha : \ \sigma = \rho\alpha \ \wedge \ \rho \in \widehat{E}_{q,v},$$

and the Parser DWA is the compiled form of this prefix test: $[\![\mathcal{P}_{\mathrm{DWA}}]\!](\rho)_{qv} = 1 \Leftrightarrow \rho \in \widehat{E}_{q,v}$. The mask query uses this equivalence by walking the represented stack prefixes through the Parser DWA.

6.2.1 Shared GSS Walk

The runtime then walks the active graph-structured stacks through the Parser DWA. Suppose a work item has reached Parser-DWA state x with accumulator A. The stack prefix read so far contributes $A \otimes \varphi(x)$ to the output mask. If the next stack symbol is p and $\tau(x, p) = (y, W)$, then the successor work item has state y and accumulator $A \otimes W$. Undefined transitions and empty accumulators are discarded. The query unions all contributions over all prefixes of all represented stacks and all active lexer-state entries. The resulting $Q_{\mathrm{lex}} \times \mathcal{V}$ mask is projected to a vocabulary mask by keeping token v whenever v appears in any lexer-state row.

The graph-structured stack implements this prefix walk without enumerating every stack. When several paths reach the same GSS node and Parser-DWA state, their accumulators are unioned with $\oplus$ before the shared suffix is explored. This preserves the semantics above: the remaining computation depends only on the GSS node, the Parser-DWA state, and the accumulated surviving token pairs.

Each branch contributes only its own q-row; carrying the full $Q_{\mathrm{lex}} \times \mathcal{V}$ domain lets a single Parser DWA serve all lexer states at once, and the final projection unions the surviving rows.

In Example 2.1, after committing `[` the runtime holds a short stack prefix at lexer state q_0. Walking that prefix through the Parser DWA accumulates the row that admits exactly {`[`, `x`, `[[`, `x]`, `[x`}, the second line of the table in Example 2.1, while `]` and `]]` receive empty weights along the walk and are dropped.

6.2.2 Exclusion Filtering

First account for longest-match exclusions. The runtime uses the precomputed lexer-match table

$$M(q, t) = \{\, v \in \mathcal{V} : t \in \mathrm{dom}(\mu_{q,\beta(v)}) \,\}, \qquad \mathrm{Match}(q, \beta(v)) = (\mu_{q,\beta(v)}, -).$$

Thus $M(q, t)$ contains exactly the vocabulary tokens whose strings match terminal t when scanned from lexer state q. For a branch with longest-match exclusion map e, define

$$X(e) = \mathcal{V} \setminus \bigcup_{q' \in \mathrm{dom}(e)} \ \bigcup_{t \in e(q')} M(q', t).$$

A token lies in $X(e)$ exactly when it passes the longest-match exclusion check for that branch. For a branch under lexer state q, the initial accumulator is the mask $A \in K$ whose q-row is $X(e)$ and whose other rows are empty.

The same branch-local filter is applied before the Parser-DWA contribution is projected to vocabulary tokens. Exclusion filtering is therefore a mask-side operation: it removes tokens whose bytes would prove that an earlier tentative terminal was emitted too early under longest-match lexing.

6.3 Commit

COMMIT updates S after a byte fragment b has been accepted. If S represents the configurations reachable after prefix u, then after committing b the new state should represent the configurations reachable after prefix ub. During ordinary decoding $b = \beta(v)$ for the sampled token v; during prefill the same operation is applied to prompt bytes or partial-completion bytes.

COMMIT starts from each active lexer-state entry in S. For each represented branch, it scans b from that lexer state. Scanner branches that enter the rejecting state are discarded. Successful scanner branches may emit zero or more completed terminals and may end either at the boundary state q_0 or in a non-boundary lexer state.

Longest-match exclusions are checked during this scan. If the branch carries $t \in e(q')$, and scanning the fragment from lexer state q' matches terminal t, then that branch is discarded. Otherwise the longest-match exclusion map is carried forward along the scanner branch, with entries updated to refer to the lexer state reached after the scanned fragment.

COMMIT then processes the completed terminal matches encountered while scanning b. For each emitted terminal t, it advances the corresponding graph-structured stacks according to the stack-effect relation E_t. Each applicable rewrite $u \xrightarrow{t} v$ replaces stack prefix u by stack prefix v, preserving the remaining stack suffix. If no rewrite applies, that branch is discarded; if several apply, the graph-structured stack represents the resulting branches and shares their common suffixes.

If scanning ends at q_0, the emitted terminal sequence is complete, so the resulting stacks are stored under lexer state q_0. If scanning ends in a non-boundary state $q' \neq q_0$, the fragment has ended in the middle of a possible terminal. The branch is retained only if some terminal in $\mathrm{CanMatch}(q')$ can still be parsed from the current stacks. This is the commit-time version of the completion check used in $\mathrm{Lex}(q, b)$: partial lexer matches are kept only when they can still lead to a valid parser continuation. Finally, if a branch emits terminal t while the same match can still continue to a lexer state q', the emitted branch records the longest-match exclusion $t \in e(q')$; if a later fragment scanned from q' encounters t, that branch is discarded, which enforces longest-match lexing.

7 Correctness

This section collects the proof obligations behind exact masking. The statement is intentionally modular: lexer execution, parser stack effects, read projection, Parser-DWA evaluation, mask aggregation, and commit preservation each correspond to one construction used earlier.

7.1 Main Theorem

Theorem 7.1 (Exact runtime masks)**.** *Let a grammar and vocabulary satisfy the finiteness conditions of §8. If the runtime state represents exactly the viable configurations after consumed byte prefix u, then* MASK *returns* $\mathrm{Mask}_L(u)$*. After* COMMIT *of any accepted byte fragment b, the updated state represents exactly the viable configurations after ub.*

The proof is by composition of the lemmas below. The mask half reduces token validity to lexer outcomes and read-projected stack prefixes; the commit half is an induction over accepted fragments.

7.2 Lexer Correspondence

Lemma 7.2 (Lexer correspondence)**.** *For any lexer state q and byte fragment b, the relation* $\mathrm{Scan}(q, b)$ *enumerates exactly the terminal sequences emitted inside b together with the lexer state left at the fragment boundary. The completed relation* $\mathrm{Lex}(q, b)$ *is exactly the set of complete terminal sequences that may be relevant to parser checking after accounting for partial matches.*

This follows directly from the two clauses defining Scan: leaving a non-rejecting suffix unemitted, or emitting a nonempty accepted terminal prefix and restarting at the boundary state. The partial-match clause adds precisely the terminals in CanMatch when the fragment ends away from a terminal boundary.

7.3 Stack-Effect Correspondence

Lemma 7.3 (Stack-effect correspondence)**.** *For each terminal t, the language E_t contains exactly the signed words $u^- v^+$ representing stack-prefix rewrites reachable by the source parser while consuming t. For a terminal sequence r, the composed language E_r contains exactly the signed words representing rewrites reachable by consuming r.*

For GLR tables, each element of E_t is a finite reduction run selected under lookahead t, followed by a shift of t, encoded as a signed word. For an ϵ-free GPDA, E_t is induced directly by the terminal-labelled transition relation. Composition with cancellation preserves precisely those adjacent effects whose pushed and read symbols agree.

7.4 Read-Projection Lemma

Lemma 7.4 (Read projection)**.** *For any terminal sequence r and parser stack σ,* $\mathrm{Parseable}(r, \sigma)$ *holds if and only if σ has a prefix in $\widehat{E}_r$.*

By definition, $\widehat{E}_r$ forgets only the replacement prefix v in a signed effect $u^- v^+ \in E_r$; it retains exactly the prefix u that must already be present on the stack before r can be consumed.

7.5 Parser-DWA Lemma

Lemma 7.5 (Parser-DWA contract)**.** *For every stack prefix ρ, lexer state q, and token v,*

$$\left[\left[\mathcal{P}_{\mathrm{DWA}}\right]\right](\rho)_{qv} = 1 \quad \Longleftrightarrow \quad \rho \in \widehat{E}_{q,v}.$$

The Terminal DWA supplies exactly the terminal sequences that token v can produce from lexer state q, together with the Boolean weights identifying those (q, v) pairs. Substitution inserts the one-terminal stack-effect automata along these terminal paths. Cancellation preserves exactly the successful composed stack effects, discarding paths with mismatched push/read pairs, and projection keeps exactly their read prefixes. Determinization preserves the resulting weighted language, while $\oplus$ unions alternative terminal sequences and alternative stack-effect paths. Hence the (q, v) coordinate accepts precisely $\widehat{E}_{q,v}$.

7.6 Mask Lemma

Lemma 7.6 (Mask computation)**.** *For any represented configuration set C, the runtime walk of §6.2 computes* $\mathrm{Mask}_{\mathrm{rt}}(C)$.

For a single branch, the initial accumulator applies longest-match exclusions; then the Parser-DWA walk contributes exactly the tokens whose required read prefix matches some prefix of the represented stack. Unioning over GSS branches and lexer-state rows is exactly the existential quantification in $\mathrm{Mask}_{\mathrm{rt}}$.

7.7 Commit Invariant

Proposition 7.7 (Runtime invariant)**.** *Let S represent the configuration set C reachable after prefix u. Then the mask query of §6.2 computes* $\mathrm{Mask}_{\mathrm{rt}}(C)$*, and after* COMMIT *of a fragment b, the updated state represents the configuration set reachable after ub. Consequently* $\mathrm{Mask}_{\mathrm{rt}}(C) = \mathrm{Mask}_L(u)$ *holds at every decoding step.*

The base case is the initial lexer state and parser start stack. For the inductive step, COMMIT mirrors lexer execution, terminal stack-effect rewrites, partial-match retention, and longest-match exclusion updates. Thus it drops no viable configuration and adds no unreachable one. Combined with the mask lemma, the invariant proves Theorem 7.1.

8 Finiteness and Scope

8.1 Bounded Terminal Effects

Lemma 8.1 (Finiteness)**.** *Suppose the grammar is normalized to remove right recursion and hidden left recursion (§8.2), and suppose the vocabulary is fixed. Then the token-level read-prefix languages $\widehat{E}_{q,v}$ have bounded depth, and the Parser DWA has bounded stack-read depth.*

Consuming one terminal must have a finite, bounded stack effect. In the GLR implementation, grammar normalization gives this property. A GPDA source may also provide it directly through its finite terminal-consuming transitions.

8.2 Grammar Normalization

The compiler applies several normalization passes before stack-effect automata are built. The key step is to remove right recursion and hidden left recursion. For a GLR parser of a CFG without right and hidden left recursion, Aycock et al. [1] show that the number of consecutive reductions between the shifts of two adjacent symbols is bounded by a grammar constant. This bounds the reduction runs that contribute to a one-terminal stack effect.

Other normalization passes serve more specific engineering purposes. Nullable nonterminals are eliminated to avoid zero-length reductions, and long productions are split so that reductions pop through several shallower advance steps rather than one deep pop concentrated into a single step.

8.3 Why the Read Depth Is Bounded

The boundedness argument has three parts.

First, each one-terminal stack effect has bounded read and write prefixes. In the GLR construction, an element of E_t is a bounded reduction run chosen with lookahead t, followed by one shift of t.

Hence every signed word $u^- v^+ \in E_t$ has bounded read and write length. In a finite ϵ-free GPDA source, the same property follows from the finite transition relation: each transition consumes one terminal and rewrites finite stack strings.

Second, token scans are finite. $\mathrm{Lex}(q, \beta(v))$ is computed by running the lexer DFA over the fixed byte string $\beta(v)$, so each token induces finitely many complete terminal sequences, each of bounded length.

Third, composition and cancellation do not introduce unbounded reads. Composing the bounded terminal effects along a token-induced terminal sequence concatenates their signed words and applies only the reductions $p^+ p^- \to \epsilon$ and $p^+ q^- \to \bot$; neither rule lengthens a word. The read prefix of a composite effect is therefore bounded by the number of terminals in the token sequence times the one-terminal read bound. The read projection $\widehat{E}_r$ keeps only those bounded prefixes. Thus every $\widehat{E}_{q,v}$ has bounded depth, and the Parser DWA reads only a bounded stack prefix at runtime.

8.4 ϵ-Moves

If a source parser contains ϵ-moves, those moves must be summarized before this interface applies. The compiler consumes one-terminal stack effects, so stack changes that occur without consuming a terminal must be folded into the surrounding terminal-consuming effects. The summarized effects must again have a finite, bounded representation.

8.5 Unsupported Parser Models

The construction assumes an incremental parser whose configurations represent prefixes that can still be extended to complete strings. A bare PDA or prefix-rewriting transition system may recognize the same complete language while exposing nonviable intermediate configurations; such configurations should be trimmed before effects are used. Parsers with unbounded terminal effects, unsummarized ϵ-moves, dynamically changing vocabularies, or context-sensitive side conditions such as type checking and uniqueness are outside the exact finite construction described here.

9 Implementation

9.1 GLR Tables

The implementation uses GLR parsing over IELR(1) tables rather than LALR(1) tables, which keeps the language-recognition power of canonical LR while retaining much of the state merging that makes LALR compact [5]. The payoff is in the straddling-terminal check of Commit: a candidate completion $t \in \mathrm{CanMatch}(q')$ can be tested by inspecting the parse-table cell for the current top LR state and lookahead t. A shift is sufficient, and a reduce action is sufficient as well: with LR(1)-precise reduce lookaheads, a reduction listed under t is on a viable path for that same lookahead and will expose another justified action, eventually reaching a shift of t. With LALR-style lookahead merging, the same table-cell test could admit a spurious reduce action; ruling that out would require an exploratory execution of parser actions.

9.2 Stack-Effect Automata

The compiler builds stack-effect automata from terminal-labelled GLR action runs. Each accepting path records a read prefix and the corresponding rewrite template used by Commit. During construction, signed-word cancellation is applied eagerly so that pushed-and-read internal symbols do not survive into the read projection.

9.3 Lexer Tables

The lexer DFA is compiled into tables for byte transitions, accepted terminals, CanMatch, and the vocabulary-indexed match table $M(q, t)$ of §6.2.2. These tables let Mask apply longest-match exclusions without rescanning the whole vocabulary.

9.4 Parser DWA Layout

Substitution groups Terminal-DWA edges by common target before inserting stack-effect recognizers, sharing subgraphs before cancellation and shrinking the NWA that must be determinized. The final Parser DWA is stored as a transition table plus final weights; each state represents a set of read-side obligations and the surviving q, v weights at that point.

9.5 Weight Representation

The compiler never materializes weights as dense $|Q_{\text{lex}}| \times |\mathcal{V}|$ Boolean arrays. Each weight is stored as a `range-set-blaze` range map from lexer-state ranges to interned token range sets. Boolean operations are therefore costed by the number of stored ranges rather than by the numeric size of the covered sets.

The compiler also constructs internal ID spaces for lexer states and tokens and may renumber them using the whole collection of weights that will be stored or queried together. The goal is not a small maximum ID, but a small number of ranges across the resulting weights and lexer-match tables. Interning both the outer maps and the inner token sets lets repeated weights be stored and compared by pointer identity. This interning turns mask equivalence into sharing: tokens whose stack consequence is identical collapse onto one stored object, which is the mechanism behind the vocabulary quotient and the resulting artifact size reported in §12. Anecdotally, vocabulary equivalence analysis is crucial for keeping compilation times practical.

After determinization, an optional acyclic minimization pass removes further redundancy. The compiler may skip it when the objects produced by substitution, cancellation, and determinization are already small enough.

9.6 Transition Tables

The Parser DWA is stored as a transition table, with default transitions for wildcard stack reads. The runtime uses an explicit transition when one matches the current stack symbol, and otherwise uses the default.

9.7 GSS Traversal

The same leveled-GSS data structure is reused with different branch annotations: longest-match exclusion maps during ordinary runtime state and dense mask accumulators during a mask query. This lets the query reuse GSS sharing while changing only the per-branch payload. In practice, the details of the GSS implementation have a large effect on runtime performance.

9.8 End-of-Sequence

The ordinary vocabulary mask covers byte-emitting tokens governed by $\beta(v)$. The stop token eos is computed separately: its bit is set exactly when the consumed prefix is itself a complete string, $u \in L$. Operationally, a configuration contributes EOS only when the lexer is at the terminal-boundary state q_0 and the parser stack is accepting on end of input. The q_0 requirement prevents stopping in

the middle of a partial terminal; the parser acceptance check prevents stopping when the current prefix only has a valid continuation.

9.9 Commit and Prefill

COMMIT is defined for arbitrary byte fragments. Committing an LLM token looks up its byte string and commits those bytes. Prefill uses the same operation directly on the input bytes, without first tokenizing them into LLM tokens.

9.10 Artifact Size

The artifact size is driven by grammar size, vocabulary quotienting, and the number of distinct weights after determinization and minimization. For practical grammars, contiguous token IDs collapse into ranges, and repeated weights are shared, keeping typical artifact size in the tens of megabytes.

10 Evaluation

We report the retained 14 October 2026 native AWS run for Static (FastRuntime), Dynamic (FastBuild), Balanced (O2), and llguidance. The run uses GLRMASK revision `5657d277` and llguidance revision `f0971424`, with the Llama 3.1 8B Instruct vocabulary of 128,256 entries. Measurements are from one native wall-clock timing pass on an AWS c8i.8xlarge, Linux x86_64, Intel Xeon 6975P-C, with 32 available logical CPUs. Vocabulary preparation is excluded. One problem runs at a time, with a 60-second construction timeout and 120-second runtime timeout. A synthetic global warmup is enabled; real schemas are not warmed up. TBM runs from committing the token allowed by the previous mask through the next mask being ready. TTFM runs from schema-dependent construction start through the first ready mask. EOS handling is included inside native ready-mask timing; other model-control tokens are excluded unless explicitly constrained as exact IDs. Percentiles use linear interpolation; means are arithmetic.

Timing qualification. The run did not pin CPUs, isolate cores, or reserve SMT siblings. Checkpoint staging, hash verification and publication overlapped the living benchmark child. Retained telemetry cannot separate intrinsic work from scheduling, steal-time or frequency effects. These observations do not certify latency stabilization or dedicated hardware isolation. Cross-machine changes relative to earlier Mac or Linux runs are not causal code-regression evidence.

10.1 JSONSchemaBench

The run attempts all 9,558 official JSONSchemaBench schemas [10], revision `ba103c73`, with 35,339 supplied validation cases; 1,072 schemas have no supplied case. Schema payloads are unchanged. Object properties are recursively reordered into schema-declaration order and separators use comma/colon followed by one space. The frozen native preparation records 4,588 reordered tests.

TTFM uses the 8,326 schemas built successfully by every engine. Headline TBM follows the current native `make jsb` reporter: 11,277 identical, complete valid trajectories accepted by every engine, comprising 1,734,930 positions per engine. Incorrect, incomplete and runtime-error trajectories are excluded from headline TBM, while all full-run outcomes remain in the raw records and are reported separately. This is a finite-fixture timing cohort, not a claim of full-language equivalence or whole-corpus dominance.

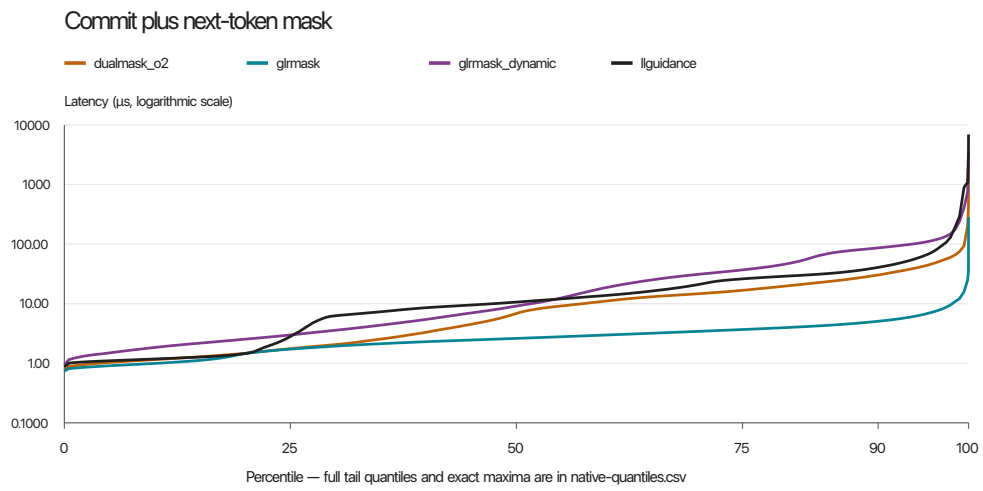


Figure 1: JSONSchemaBench native headline TBM, 1,734,930 positions per engine from 11,277 complete valid trajectories accepted by all four engines. One unpinned wall-clock pass; checkpoint work overlapped the benchmark.

Static median TBM is 2.61 μs and p99 is 12.15 μs, compared with 10.63 μs and 289.92 μs for llguidance. Static's measured maximum is 283.07 μs. The same token position is a recurring hotspot in the same-source October 11 Linux run; the retained evidence does not isolate algorithmic and scheduling contributions. Dynamic and Balanced retain separate runtime/build tradeoffs in Tables 1 and 2.

Table 1: JSONSchemaBench TBM in microseconds, token-weighted over the native complete-valid common cohort. Linear-interpolated percentiles.

System	Median	p90	p99	p99.9	Maximum
GLRMask Static	2.61	5.07	12.15	24.94	283.07
GLRMask Dynamic	9.12	86.41	239.52	754.22	3,647.84
GLRMask Balanced	6.86	30.31	74.77	206.12	2,140.92
llguidance	10.63	40.67	289.92	1,092.63	6,931.51

Table 2: JSONSchemaBench TTFM in milliseconds over common-built schemas. One construction observation per schema and engine in this run.

System	Median	p90	p99	p99.9	Maximum
GLRMask Static	12.57	78.87	302.30	854.35	1,293.39
GLRMask Dynamic	2.30	6.54	35.85	88.81	274.62
GLRMask Balanced	6.23	18.96	125.49	444.98	631.14
llguidance	0.75	1.82	11.11	17.90	231.01

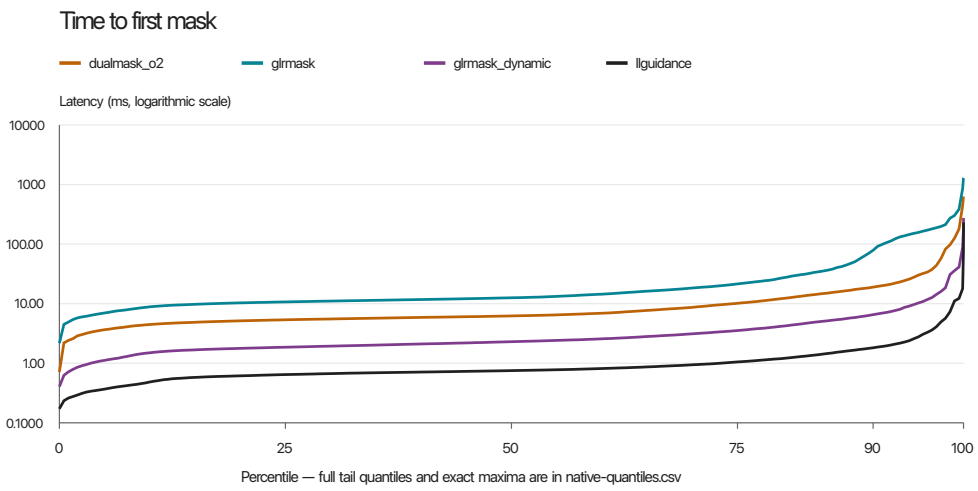


Figure 2: TTFM over 8,326 common-built schemas. A successful build does not establish semantic correctness. Vocabulary preparation is excluded.

Static's common-build TTFM maximum is 1.293389335 s. It occurs at `Github_ultra--o21135`. Construction takes 1.293215283 s. The all-supported Static maximum is 2.373731268 s, outside this common-build cohort because llguidance fails to build that schema. The two maxima describe different populations and are not substituted for one another.

Coverage and correctness. All 38,232 JSON engine/schema records remain in the retained archive, including 1,923 build errors. Incorrect examples are disagreements with expected validation outcomes. Table 3 counts every attempted schema and records outcomes outside the timing population.

Table 3: Full outcomes on 9,558 JSON schemas per engine. Build columns count schemas; incorrect and runtime-error columns count examples.

System	Build success	Build error	Incorrect	Runtime error
GLRMask Static	9,325	233	460	0
GLRMask Dynamic	9,326	232	460	0
GLRMask Balanced	9,326	232	460	0
llguidance	8,332	1,226	100	6

10.2 JavaScript

The practical-JavaScript corpus, revision `a3738a35`, contains 55 cases: 31 capability fixtures and 24 negative fixtures. All 220 engine/case builds succeed, but each GLRMask mode has one incorrect example and llguidance has four. Only 51 fixtures are correctly evaluated by every engine. No runtime-error examples occur. The companion grammar policies differ; this corpus does not establish full ECMAScript support or complete-language equivalence.

Native headline TBM uses 28 complete valid trajectories accepted by all four engines, with 3,805 positions per engine. TTFM contains 55 repeated cold-build case observations of one practical-JavaScript grammar, not 55 different grammar samples. The wall-clock protocol and timing qualifications above apply to both workloads. There is no minimum-of-20 estimator in this AWS run.

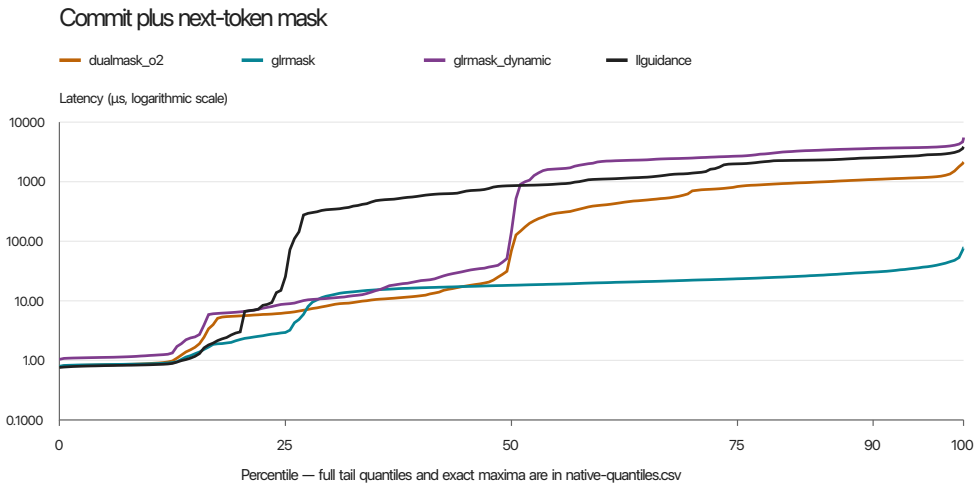


Figure 3: Practical-JavaScript native headline TBM: 28 common complete valid trajectories, 3,805 positions per engine, one unpinned wall-clock pass.

Table 4: JavaScript TBM in microseconds over the native valid-common cohort.

System	Median	p90	p99	p99.9	Maximum
GLRMask Static	18.25	30.36	47.87	71.11	77.38
GLRMask Dynamic	145.48	3,626.08	4,091.70	4,656.85	5,517.38
GLRMask Balanced	69.95	1,088.80	1,509.59	2,016.84	2,138.95
llguidance	856.61	2,521.67	3,096.99	3,597.12	3,867.91

Table 5: JavaScript TTFM in milliseconds over 55 cold-build case observations sharing one grammar. These are not independent grammar samples.

System	Median	p90	p99	p99.9	Maximum
GLRMask Static	5,099.56	5,152.72	5,223.03	5,225.88	5,226.19
GLRMask Dynamic	113.03	116.87	120.69	121.66	121.77
GLRMask Balanced	457.85	464.32	470.41	472.25	472.46
llguidance	7.97	8.05	8.16	8.20	8.20

Provenance and historical results. The complete source archive SHA-256 is `0267240e2b842ccbe4408d7144d1cc898b310468798488f630fed4df577ba7`. The corrected native exports and all error records are re-

tained. The prior October 8 JSON and October 9 JavaScript Mac evaluation source and its dated figures remain preserved under `review/paper/history/aws-update-20261014` and their original asset paths. Their hardware, calling-thread CPU runtime clock and repetition policies differ from this AWS run. No timing workload was rerun during data recovery or plot correction.

11 Related Work

Grammar-constrained decoding. Geng et al. [9] formalized grammar-constrained decoding (GCD) by defining a token as valid iff the current prefix can be extended to a string in the grammar's language (eq. (1)). The straightforward implementation tests this condition online for every vocabulary token at every step. GLRMask keeps the same semantics (§2.1) but moves the test offline.

The finite-state construction in Outlines [19] compiles a regular expression into a token-level finite automaton, after which masks are precomputed transition-table lookups. This construction supports regular constraints; unbounded nesting requires a parser. The same paper also describes a CFG extension using LALR(1) parsing. Koo et al. [12] extend automata-based decoding to deterministic context-free languages. Their token-level PDA retains runtime stack-compatible edge checks.

DOMINO [2] precomputes scanner-state subterminal tries and traverses them with an online parser. Its lookahead setting controls how far this traversal follows tokens across terminal boundaries; bounded lookahead can exclude otherwise valid tokens. XGrammar [6] compiles JSON schemas and EBNF grammars into byte-level pushdown automata. It caches masks for context-independent tokens and checks context-dependent candidates against the runtime stack. XGrammar-2 [13] adds just-in-time compilation and cross-grammar caching; context-dependent tokens are checked against the full Earley parsing context. llguidance [15] runs an Earley parser on a separate derivative-based lexer over regex terminals, traversing a vocabulary trie to construct masks. GLRMask handles token admissibility through compiled stack-reading weights.

Pre^3 [4] precomputes prefix-conditioned transitions for LR(1) grammars, enabling runtime mask construction by checking compiled stack conditions. Park et al. [16] make grammar preprocessing faster to reduce per-request compilation cost, keeping masking online.

CFGzip [17] uses offline stack-displacement analysis to compute grammar-relative token equivalence classes. It reduces the token space presented to an online grammar engine, which still performs the mask checks. This is related to GLRMask's vocabulary equivalence analysis, while GLRMask also compiles the stack-dependent mask computation. Gram2Token [11] compiles deterministic byte-level grammar execution into token categories, masks, and transition tables for GPU execution.

Parser-stack classification. PSC [14] likewise moves stack-dependent mask construction into preprocessing for deterministic context-free grammars. It constructs stack-transformer transducers, combines token acceptance conditions into a deterministic finite-state classifier, and obtains a precomputed vocabulary mask by traversing the current parser stack. The high-level insight that a compiled finite-state structure can read the parser stack directly is therefore shared. Our representation retains Boolean weights over lexer-state/token pairs on automaton transitions and final states, permitting token-dependent information to be factored through shared stack-reading structure, rather than only classifying states into output masks. We additionally describe the construction and runtime traversal for graph-structured stacks. These are differences of representation and scope; this paper does not include a controlled head-to-head performance comparison with PSC.

Weave of Formal Thought [3] also combines generalized parsing and speculative lexical hypotheses. It validates candidate subwords by speculative parsing at runtime, whereas GLRMask queries

compiled mask weights.

Generalized parsing. GLRMask's runtime state is a graph-structured stack (GSS), the data structure introduced by Tomita [18] to handle ambiguous LR parsing. At a fork the GSS represents multiple parser configurations that share their common stack suffix; configurations that reach the same parser state at the same input position are merged. The per-target merging in our mask query (§6.2) is essentially a GSS-style sharing argument lifted from input positions to Parser-DWA states.

Aycock et al. [1] show that, for grammars without right recursion or hidden left recursion, the number of consecutive reductions between shifts is bounded by a grammar-dependent constant. GLRMask uses this result in the bounded-read-depth argument of §8.3.

The push/read cancellation algebra of §4.5 is a special case of the well-known representation of pushdown transitions as elements of a monoid with relations $P_p R_p \to \epsilon$ and $P_p R_q \to 0$ for $p \neq q$; compositions are then ordinary monoid products. The read projection is defined in §4.6. Compiling a CFG into a weighted automaton whose weights are themselves token sets fits into the more general framework of weighted automata over semirings [7, 8]: our weights form the Boolean lattice $\mathbb{B}^{Q_{\text{lex}} \times \mathcal{V}}$ under $(\vee, \wedge)$, and the runtime walk combines weights using these operations.

Comparison. Table 6 sketches the design space. The combination that distinguishes GLRMask is exact masks for normalized CFGs, including ambiguous ones, using a finite weighted automaton that reads parser stacks directly rather than simulating parser transitions at query time.

Table 6: Masking mechanisms by system.

System	Grammars	Precomputation	Runtime mask work	Ambiguity
Outlines (regular)	Regular	Token DFA	Mask lookup	—
Koo et al. [12]	DCFL	Token PDA	Stack-compatible edges	unsupported
XGrammar	CFG	Cached masks	Context-dependent PDA checks	supported
XGrammar-2	CFG	JIT/cached masks	Earley-context checks	supported
llguidance	Lexed CFG	Lexer and trie	Earley/trie traversal	supported
DOMINO	Lexed CFG	Subterminal tries	Trie-path parser checks	parser-dependent
Pre^3	LR(1)	Prefix-conditioned DPDA	Stack-condition checks	unsupported
PSC [14]	DCFG	Stack classifier	Compiled stack walk	unsupported
GLRMask (this paper)	**CFG**	**Parser DWA**	**Compiled GSS walk**	**supported**

The table summarizes masking mechanisms for the stated grammar classes; supported grammar features and lexical semantics differ across systems. The Outlines row describes its regular-constraint method.

12 Discussion

Scope and limits. The Parser DWA's size scales with grammar size and with the mask-equivalence quotient of the vocabulary. We have not characterised this dependence asymptotically or measured a regime in which the Parser DWA does not fit in RAM.

Compilation is per-(grammar, vocabulary): token bytes determine Terminal-DWA paths and Parser-DWA weights, so a tokenizer change forces a full recompile. In deployed serving stacks this is normally amortized across a fixed model and tokenizer.

Many real-world constraints are not context-free: variable-scoping rules, type-correctness, and value uniqueness in JSON schemas. These constraints are outside the scope of this construction.

Pathological ambiguous grammars can also cause stack forks that scale super-linearly in generation length.

13 Conclusion

GLRMask compiles the parser-dependent part of token admissibility ahead of time. Joint (lexer-state, vocabulary-token) indexing, push/read cancellation, and weighted determinization produce a deterministic weighted automaton over parser-stack symbols. At runtime, Mask reads the graph-structured parser stack through this automaton, while Commit applies the compiled stack rewrites after a token is sampled.

References

[1] John Aycock, R. Nigel Horspool, Jan Janoušek, and Bořivoj Melichar. Even faster generalized LR parsing. *Acta Informatica*, 37:633–651, 2001. doi: 10.1007/PL00013319.

[2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 3658–3673. PMLR, 2024. arXiv:2403.06988.

[3] Alexandre Bouayad. Weave of formal thought. *arXiv preprint arXiv:2606.25987*, 2026. URL `https://arxiv.org/abs/2606.25987`.

[4] Junyi Chen, Shihao Bai, Zaijun Wang, Siyu Wu, Chuheng Du, Hailong Yang, Ruihao Gong, Shengzhong Liu, Fan Wu, and Guihai Chen. Pre^3: Enabling deterministic pushdown automata for faster structured LLM generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11253–11267. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.551.

[5] Joel E. Denny and Brian A. Malloy. The IELR(1) algorithm for generating minimal LR(1) parser tables for non-LR(1) grammars with conflict resolution. *Science of Computer Programming*, 75(11):943–979, 2010. doi: 10.1016/j.scico.2009.08.001.

[6] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100*, 2024.

[7] Manfred Droste, Werner Kuich, and Heiko Vogler, editors. *Handbook of Weighted Automata*. Springer, 2009. doi: 10.1007/978-3-642-01492-5.

[8] Uli Fahrenberg and Axel Legay. Featured weighted automata. *arXiv preprint arXiv:1702.07484*, 2017.

[9] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952. Association for Computational Linguistics, 2023.

[10] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*, 2025.

[11] Hantao Hua, Jiming Su, Hao Tang, Yiping Yao, and Feng Zhu. Gram2Token: Enabling run-time GPU-native grammar-constrained decoding for LLMs. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 46169–46185. PMLR, 2026. URL `https://proceedings.mlr.press/v306/hua26b.html`.

[12] Terry Koo, Frederick Liu, and Luheng He. Automata-based constraints for language model decoding. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2407.08103.

[13] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2:

Dynamic and efficient structured generation engine for agentic LLMs. *arXiv preprint arXiv:2601.04426*, 2026. URL https://arxiv.org/abs/2601.04426.

[14] Yongmin Li, Yihong Dong, Jia Li, and Ge Li. Efficient grammar-constrained decoding via parser stack classification. *arXiv preprint arXiv:2608.03065*, 2026. URL https://arxiv.org/abs/2608.03065.

[15] Microsoft. llguidance: Low-level guidance library for constrained decoding. https://github.com/guidance-ai/llguidance, 2024. Core engine for constrained generation in Guidance.

[16] Kanghee Park, Timothy Zhou, and Loris D'Antoni. Flexible and efficient grammar-constrained decoding. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 48262–48275. PMLR, 2025. URL https://proceedings.mlr.press/v267/park25l.html.

[17] Michael Sullivan and Alexander Koller. Accelerating constrained decoding with token space compression. *arXiv preprint arXiv:2605.29986*, 2026. URL https://arxiv.org/abs/2605.29986.

[18] Masaru Tomita. An efficient context-free parsing algorithm for natural languages. In *Proceedings of the 9th International Joint Conference on Artificial Intelligence*, pages 756–764, 1985.

[19] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.